SAVEETHA SCHOOL OF ENGINEERING

OPERATING SYSTEMS

ASSIGNMENT | CO1

EVALUATE AND JUSTIFY THE OPERATING-SYSTEM DESIGN OF A SMART HOSPITAL

Student Details	Information
Name	S. Maruthu Pandi
Register Number	192511162
Department	B.E. Computer Science and Engineering (Cyber Security)
Course	CSA0421 – Operating Systems
Assignment	Assignment 1 (Tasks 9)

TASK 9 – EVALUATE THE MEMORY HIERARCHY AND I/O SYSTEMS

9.1 Memory hierarchy

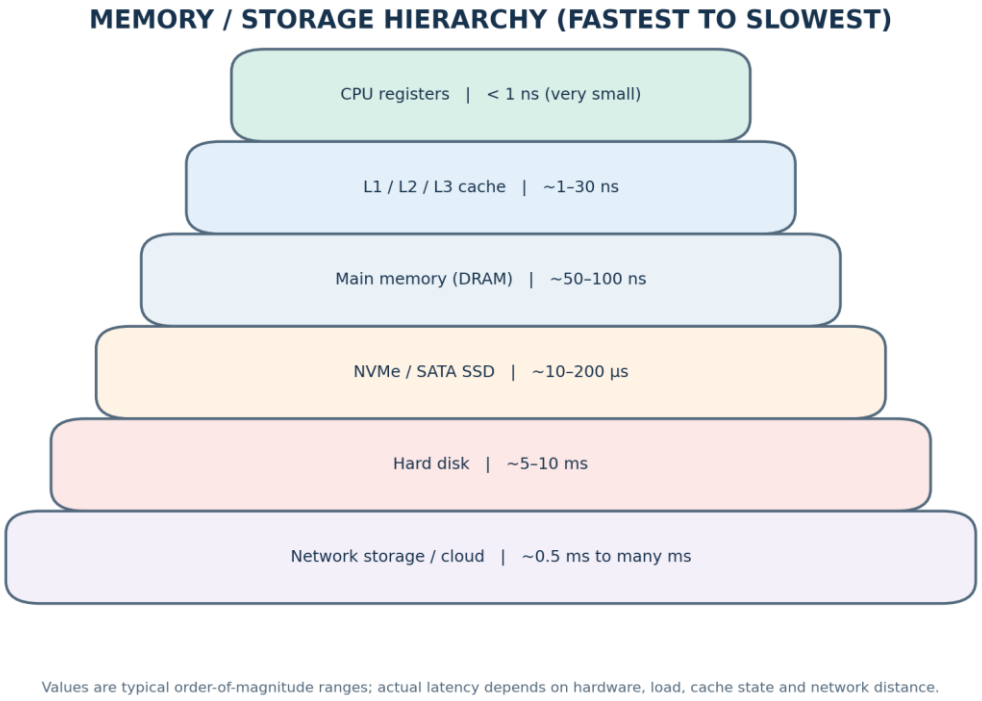


Figure 6. Storage becomes larger and cheaper per byte as it moves down the hierarchy, but access latency increases.

Level	Typical access time (approx.)	Typical capacity scale	Hospital use
CPU registers	Below 1 ns	Bytes to a few KiB per core	Current instruction operands, counters and immediate alarm calculations.
L1 / L2 / L3 cache	About 1–30 ns	Tens of KiB to tens of MiB	Frequently used code/data and hot database or image-processing working sets.
Main memory (DRAM)	About 50–100 ns	Several GiB to many hundreds of GiB	Active patient-record cache, monitor buffers, image reconstruction data and process memory.

Level	Typical access time (approx.)	Typical capacity scale	Hospital use
NVMe / SATA SSD	About 10–200 µs (device/workload dependent)	Hundreds of GiB to many TiB	Database files, transaction logs, active imaging data and frequently accessed records.
Hard disk	About 5–10 ms average mechanical access order	Several TiB and above	Large image archives, historical data and backup repositories where latency is less critical.
Network storage / cloud	About 0.5 ms to many ms; may be much higher over WAN	Scalable across servers / sites	Replicated records, shared hospital data, disaster recovery and long-term archive.

These are indicative order-of-magnitude values, not guarantees. CPU generation, cache state, SSD interface, queue depth, filesystem, network path and current load can change the result substantially.

9.2 Where hospital data should be stored

Data	Primary placement	Reason / protection
MRI images	DRAM during reconstruction; fast SSD/NVMe for active studies; high-capacity archive for older images.	Large sequential data benefits from high bandwidth. Use checksums, access control, backup and lifecycle retention.
Patient records	Database memory cache plus durable SSD-backed clustered database storage and replicated/cloud backup.	Low-latency lookup needs caching; durability and availability require transaction logs, replication and tested backup.

Data	Primary placement	Reason / protection
Monitor readings	Small local RAM/ring buffer for real-time processing; durable local log or ward gateway queue; central database for long-term records.	The alarm path must not wait for network storage. Buffer data locally during temporary network outages and forward it later.

9.3 Programmed I/O, interrupt-driven I/O and DMA

Method	How it works	Best fit / limitation
Programmed I/O (polling)	CPU repeatedly checks device status and transfers data through programmed instructions.	Simple low-rate devices; wastes CPU time if the device is slow or idle.
Interrupt-driven I/O	Device interrupts the CPU when it needs service or has completed an operation.	Useful for moderate or event-driven I/O; too many interrupts can create high CPU overhead.
Direct Memory Access (DMA)	A DMA controller transfers blocks between a device and memory with limited CPU involvement; completion is reported by interrupt.	Best for high-volume MRI image transfers and other bulk I/O.

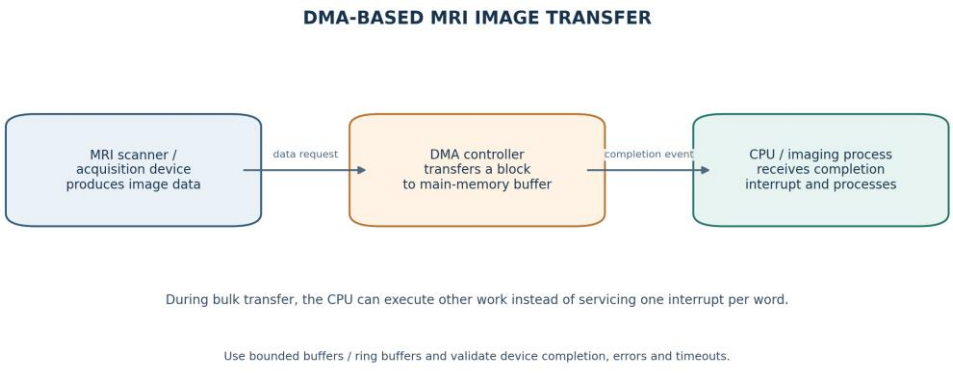


Figure 7. DMA transfers an image block into a memory buffer, allowing the CPU to process other work until completion.

9.4 Effect of caching on record lookup

A database cache keeps frequently accessed pages in main memory. If a requested record page is already cached, the database can avoid a storage read and return data much faster. If it is not cached, the database may need SSD or network-storage access. A well-sized cache improves average lookup latency and reduces storage I/O, but it does not replace durable storage or backups. Cache contents can be lost after power failure, so committed data must be made durable according to the database's transaction rules.

9.5 Alternative: interrupt-driven transfer of a 500 MB MRI image, one interrupt per word

Assume the assignment's "500 MB" is interpreted as 500 MiB (524,288,000 bytes) and each interrupt transfers one 4-byte word. The number of word transfers and therefore interrupts is 524,288,000 / 4 = 131,072,000 interrupts. If "500 MB" means decimal 500,000,000 bytes, the count is 125,000,000. Either interpretation produces an impractically large interrupt count for bulk image transfer.

For an illustrative interrupt-service cost of 1 μs per interrupt, 131,072,000 interrupts consume about 131 seconds of CPU time. At 5 μs each, the total is about 655 seconds. These are arithmetic examples, not measured service times; the actual value depends on CPU, interrupt controller, kernel and handler. The calculation demonstrates why DMA is selected for bulk transfer.

With DMA, the device can transfer large blocks into one or more bounded buffers and generate an interrupt on completion. For example, a double-buffer arrangement with two 1 MiB buffers uses approximately 2 MiB of transfer-buffer memory while allowing one buffer to be consumed as the next is filled. A real MRI pipeline may use a larger ring buffer based on throughput and processing latency. Interrupt-driven I/O still needs buffers, but it would require vastly more CPU intervention if one interrupt is generated per word.